THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

Written Assignment 1

Q1. Write a C program to merge two one dimensional arrays.

A1.
```
#include <stdio.h>

void main() {
    int c1= 3, c2=4;
    int arr1[] = {2, 1, 4};
    int arr2[] = {5, 7, 9, 3};

    int ans[c1+c2];
    for(int i=0; i<c1; i++) {
        ans[i] =arr1[i];
    }
    for (int i= 0; i<c2; i++) {
        ans [c1+i] = arr2[i];
    }
    printf("Merged Array: ");
    for(int i=0; i< c1+c2; i++){
        printf("%d ", ans[i]);
    }
}
```

Q2. Explain in detail multiple stack with diagrams and write a C program for two way stack.

A2. A multiple stack is a concept where more than one stack is implemented using a single array.
It allows us to dynamically allocate the size of each of the sub arrays ~~each~~ ~~A~~
There are 2 ways to implement multiple stack. 2 way stack & Multiple stack.

2 way stack has elements from both ends of the array and each of the stack can dynamically resize each other depending on their needs. ~~They~~ One array is indexed from 0 to the max size and the other is from max size to zero. ~~This~~ All overlaps are to be taken into account.

1	4	5	6	7	8	3	2	4	5

Array 1 Array 2

This shows how the size of each array can be dynamically allocated.

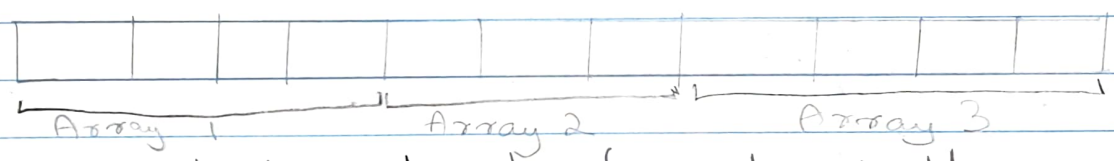
Pushing to array 1 - moves its end one space towards the right and ~~a~~ ~~gets the number to~~ store the number
Popping array 1 - moves its end one space towards the start of the array.

~~Popp~~Pushing to array 2 - Moves its end one space towards the start and stores the number.

Popping array 2 - Moves its end one space towards the end of the array.

Multiple stack.

It divides the array into 3 or more parts and store the start and ~~number of~~ ending indexes of the ~~arrays~~ individual sub arrays.



The start and end of each of the ~~arrays~~ indexes of each of the arrays is stored.

The start values remain constant of all and are used when the arrays are displayed.

The ends are ~~added~~ incremented and decremented as and when push and pop are called respectively.

THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

C program for two way stack

```
#include <stdio.h>
void pushs1 (int s[], int *c1, int *c2, int num) {
    if ((*c1) <= (*c2)) {
        s[*c1] = num;
        (*c1)++;
    } else {
        printf("Error\n");
    }
}

void pushs2 (int s[], int *c1, int *c2, int num) {
    if ((*c1) <= (*c2)) {
        s[*c2] = num;
        (*c2)--;
    } else {
        printf("Error\n");
    }
}

int popS1 (int s[], int *c) {
    ~~int val = s[*c];~~
    ~~(*c)--;~~
    ~~return val;~~
    if ((*c) > 0) {
        int val = s[*c]; (*c)--; return val;
    } else {
        printf("Error\n");
    }
```

THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

```
int popS2(int s[], int *c) {
    ~~int val = s[*c]; if((*c)<size-1){~~
    ~~(*c)++;~~ int val = s[*c]; (*c)++; return val;
    ~~return val;~~ } else {
        printf("Error\n");
    }

void display(int s[], int s1, int s2, int size){
    for(int i=0; i<s1; i++){
        printf("%d ", s[i]);
    }
    for(int i = size-1; i>s2; i--){
        printf("%d ", s[i]);
    }
}
void main(){
    int size = 5;
    int s[size];
    int s1t = 0, s2t = size-1;
    pushs1(s, &s1t, &s2t, 10);
    pushs1(s, &s1t, &s2t, 11);
    pushs1(s, &s1t, &s2t, 12);
    push s2(s, &s1t, &s2t, 13);
    pops1(s, &s1t);
    pop s2(s, &s2t, size);
    display(s, s1t, s2t, size);
}
```

THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

Q3. Describe multiple queues with suitable digrams.

Ans. Multiple queus mean two or more queues are maintained simultaneously, elements are distributed between them and they all are separate independent ~~ob~~ instances of the main struct. Each queue operates indepently with common insert, delete and display functions, but the queue ~~names are passed~~ to be modified pointer is passed as function argments to the function.

Queue1 -

Queue 2 -

Queue 3

Each queue can have different sizes ~~as an~~ depending on the developers need.
Each queue has its array, front and rear stored.

There are 3 standard functions of enqueue, dequeue and display.
It ~~moves~~ moves start right when dequeue is called and end to the right when enqueue is called.
Display iterates from start to ending indexes of the queue.